ERS V2 - eSports Arena Manager

DSY1104 | Evaluación Parcial 2 | Versión de entrega del 17 de octubre de 2026

1. Propósito y alcance

Especificar el comportamiento implementado para la demostración de torneos eSports en React. El proyecto mantiene la propuesta de EP1 y sus juegos: League of Legends, Valorant, Ajedrez, Rocket League y Minecraft. El repositorio Fullstack2-React se consultó como referencia de componentes y pruebas; no se reemplazó con el proyecto eSports.

La solución funciona en un navegador, con datos iniciales y persistencia local. Las referencias a comercio de la pauta se interpretan como contexto general de la evaluación; este ERS conserva eSports y deja explícita la validación docente del caso. Los requisitos de este documento describen la versión implementada, no requisitos nuevos aprobados por el docente.

2. Actores

Actor simulado	Capacidades
Visitante	Consultar inicio, listado y detalle; sin operaciones de escritura
Jugador	Editar su perfil, crear equipos propios e inscribirse individualmente
Capitán	Es un jugador responsable de un equipo; puede gestionarlo e inscribirlo
Organizador	Gestionar torneos; conserva identidad de jugador para otras operaciones
Administrador	Gestión general, excepciones de propiedad y restablecimiento confirmado

Los perfiles se seleccionan desde el encabezado y no acreditan identidad. Ninguna restricción de esta demostración debe considerarse autorización de seguridad de un servidor.

3. Requisitos funcionales y aceptación

ID	Requisito implementado	Criterio de aceptación
RF-01	Consultar inicio y navegar	Se muestran resúmenes y cierres abiertos vigentes; enlaces llevan a rutas válidas
RF-02	Buscar y filtrar torneos	Combina nombre, juego, estado y fechas; rango invertido informa error y permite limpiar
RF-03	Consultar detalle por URL	id identifica torneo; se muestran participantes, rondas, ranking y premios; ID desconocido informa ausencia
RF-04	Inscripción individual	Jugador propio o administrador confirma; se agrega una vez y se actualizan cupos e historial
RF-05	Inscripción por equipo	Capitán o administrador inscribe equipo activo, del juego y con integrantes completos sin sanciones
RF-06	Cancelar inscripción	Participante autorizado cancela dentro del plazo y libera el cupo; una inscripción ajena se rechaza
RF-07	Gestionar equipos	Crear, editar y eliminar con validaciones; no se modifica un equipo inscrito o histórico
RF-08	Editar perfil	Guarda nombre, apodo único y correo válido; los cambios sobreviven a una recarga
RF-09	Gestionar torneos	Organizador/administrador crea, edita o elimina si no hay participantes ni partidas
RF-10	Conservar datos	Se recupera versión 2; JSON corrupto genera aviso y semilla nueva; un fallo al guardar no publica cambios
RF-11	Restablecer demostración	Solo administrador, con confirmación, sustituye los datos locales por la semilla
RF-12	Recuperar historia Rocket League	La semilla y la migración usan equipos 2v2; preservan perfiles y otros equipos guardados

4. Reglas de negocio

- Inscripción: torneo abierto, plazo vigente, cupo positivo y participante no duplicado. El día de cierre sin hora incluye hasta las 23:59:59.999 de la zona local del navegador.
- En equipos se exige el tamaño exacto del juego al inscribir. Se permiten borradores de uno a ese máximo. Los integrantes existen, no se repiten y el capitán pertenece al equipo. El creador no administrador debe ser su capitán.
- Nombre de equipo: 3 a 60 caracteres, sin duplicados ignorando mayúsculas. Nombre de perfil: 2 a 80; apodo: 3 a 20 letras, números, guiones o guiones bajos, sin duplicados. El correo debe tener una estructura válida.
- El equipo con inscripción o historial queda protegido frente a edición y eliminación. No se retiran referencias históricas para permitir borrados.
- Torneo: nombre de 3 a 80 caracteres, juego válido, cupo entero entre 1 y 128, cierre no posterior al inicio y fechas válidas. No puede reducir cupos por debajo de sus inscritos ni cambiar el juego si ya tiene participantes.
- Estados de torneo: abierto, en-curso y finalizado. No se permite retroceder estados. Un torneo abierto requiere cierre vigente. Cambiar estado no genera partidas automáticamente.

- El servicio escribe la siguiente versión en almacenamiento antes de notificar a React. Si falla la escritura, no informa éxito ni modifica el estado publicado.
- Reiniciar reemplaza datos locales, incluidas las fechas de demostración. La migración de Rocket League no se repite tras una recarga correcta.

5. Modelo de información

Entidad	Datos principales y relaciones
Juego	ID, nombre, modalidad e integrantes por equipo
Jugador	ID, nombre, apodo, email y sanciones
Equipo	ID, nombre, juegoId, capitanId, integrantes y estado activo
Torneo	ID, juegoId, estado, fechas, cupo y referencias de participantes
Partida	Torneo, ronda, participantes, fecha y resultado histórico
Ranking y premio	Colecciones por torneo usadas en consultas

Un torneo tiene un juego y varios participantes; la modalidad del juego determina si las referencias son de jugadores o equipos. Un equipo tiene varios jugadores y un capitán. La semilla es independiente del snapshot mutable de trabajo y el estado que consume React se congela.

6. Arquitectura e interfaces

index.html carga main.jsx. App configura React Router y el actor simulado. ArenaProvider ofrece datos y servicio; crearArenaStore recibe almacenamiento, reloj y semilla, lo que permite sustituirlos en pruebas. Las páginas mantienen filtros/formularios con useState y usan props para identidad o componentes reutilizables. useSyncExternalStore sincroniza las vistas después de guardar.

Las claves locales son esports-arena-datos-v2 y esports-arena-sesion-v2. El almacenamiento depende del origen y del perfil del navegador; no se sincroniza entre dispositivos ni ofrece resolución de conflictos entre pestañas. Cambiar puerto puede mostrar otra demostración.

7. Requisitos no funcionales

ID	Objetivo	Evidencia
RNF-01	Interfaz adaptable	Siete pantallas a 360, 768 y 1366 px, con tablas desplazables
RNF-02	Mensajes y etiquetas comprensibles	Formularios etiquetados, alertas, confirmaciones y enlace de salto al contenido
RNF-03	Separar responsabilidades	Páginas, componentes, contexto, servicio y funciones puras
RNF-04	Reproducir instalación y validación	package-lock, npm ci, build, Karma/Jasmine y CI
RNF-05	Aislar pruebas	Almacenamiento y reloj inyectados; desmontaje y limpieza del DOM
RNF-06	Preservar datos ante fallos	Validaciones de estructura y persistencia atómica en pruebas de servicio

No se ha certificado conformidad WCAG, rendimiento con carga masiva ni compatibilidad universal. La validación visual se realizó en Edge Chromium y CI ejecutó Chrome.

8. Trazabilidad y exclusiones

RF-01 a RF-03: Inicio, Torneos, TorneoDetalle y tests/componentes.spec.jsx. RF-04 a RF-06: Inscripcion, servicio y pruebas de componentes, servicio y límites. RF-07: Equipo y CRUD de servicio. RF-08: Perfil e historial. RF-09 y RF-11: Administracion, guardas de rol y pruebas de confirmación. RF-10 y RF-12: semilla, servicio y pruebas de persistencia, fallos y migración. RNF-01: capturas y navegador.json. RNF-04/RNF-05: configuración Karma, workflow y TESTING.md.

Fuera de esta versión: backend, autenticación real, registro de usuarios, notificaciones, pagos, generación automática de llaves, edición de resultados y sincronización multiusuario. Los datos de partidas, rankings, premios y estadísticas de demostración no equivalen a información deportiva real. La entrega no acredita la exposición individual ni la aprobación del caso por el docente.

9. Documentos de referencia

PLAN-DE-TRABAJO-EP2.pdf, 7 páginas; pauta DSY1104 Evaluación Parcial 2_EE_EP_Estudiante.pdf, 9 páginas; Anexo 1 Instrucciones, 12 páginas. Se utilizaron como criterios de revisión. La autorización de cambios, ramas y espera entre commits provino de la solicitud expresa del usuario. La disponibilidad del ZIP Interfaz Pro mencionado en el plan no se presume: no fue adjuntado.